



CS5224 Cloud Computing

Preliminary Report

27 Sep 2026

Group 12

Name	Matric Number
DONG LANGQING	A0187171B
HORINOUCHI HIKARU (RAWIN)	A0330350X
HSIEH MING CHENG	A0351161R
KU WEE KIAT	A0307238E
RAMALINGAM SARAVANAMANI	A0217550L
SHYAM GANESH JAYAGOPI	A0218448X

FlatSplit — SaaS Cloud Platform for Group Flat Hunting

1. Selected Theme

In Singapore, students and young working adults often share an HDB flat to reduce housing costs. Choosing where to rent is difficult when flatmates have different campuses or workplaces, budgets, and acceptable commute times. Existing rental tools help users check rents or browse individual listings, but **do not compare areas against the needs of an entire group**.

FlatSplit is a SaaS platform for groups searching for a shared flat. Each member enters their destination, budget per person, and maximum commute time. The platform ranks suitable areas using **estimated rent per person and each member's public-transport commute**, while highlighting options that place a disproportionate travel burden on one flatmate.

2. Business Model

FlatSplit uses a freemium model. The free service helps groups compare rental areas and make a shared decision; paid features could support renters who want to monitor their options over time and professionals who advise clients. Paid features are future opportunities, not part of the project prototype.

- **Target customers:** Groups of university students and young working adults renting together in Singapore. Property agencies and relocation providers are potential business customers.
- **Free service:** Flatmates enter their budgets and destinations, receive an area-level shortlist, compare each person's commute, and share the results with their group.
- **Potential revenue:** A Group Pass could offer saved searches, rental trend comparisons, and rent alerts. Professional subscriptions could provide client comparison reports.
- **Customer acquisition:** University communities, housing-related content, and referrals when flatmates invite others into a group search.
- **Key costs:** Cloud infrastructure, rental and transport data processing, product development, and customer support. Cached routes and serverless services help manage costs as usage changes.

3. Cloud Service to be Developed

- **System Scope:** FlatSplit's prototype has three parts: a group session flow where 2–5 flatmates submit their destination, budget, and max commute via a shared link; a batch layer that ingests HDB's monthly rental data and precomputes public-transport commute times (via OneMap and LTA DataMall) between towns and common destinations; and a ranking layer that filters candidate towns against each member's constraints and scores them on rent, average commute, and commute fairness. Scope is limited to public-transport estimates, historical (not live) rental prices, town-level recommendations rather than individual listings, and Singapore destinations only; there is no login, payment, or booking in this version.
- **Core Microservices Architecture:** FlatSplit uses a serverless, event-driven architecture on AWS. User-facing requests are served by two Lambdas behind API Gateway, groups-api (session creation, input collection) and search-api (filtering and fairness-based ranking) which are both backed by DynamoDB for low-latency reads of precomputed rent and commute data. Data preparation is decoupled from this path: EventBridge triggers a monthly ingestion

Lambda that cleans the HDB rental dataset (raw data in S3, stats in DynamoDB) and fans out town-destination pairs via SQS to commute-worker Lambdas, which call OneMap/LTA DataMall and write the resulting commute matrix back to S3/DynamoDB. This split lets the heavy, parallel batch work scale to zero between monthly runs while the lightweight query path scales independently on demand, the core of FlatSplit's cloud-elasticity argument against an on-premise deployment, which would need fixed capacity for an idle monthly spike.

4. Preliminary Design

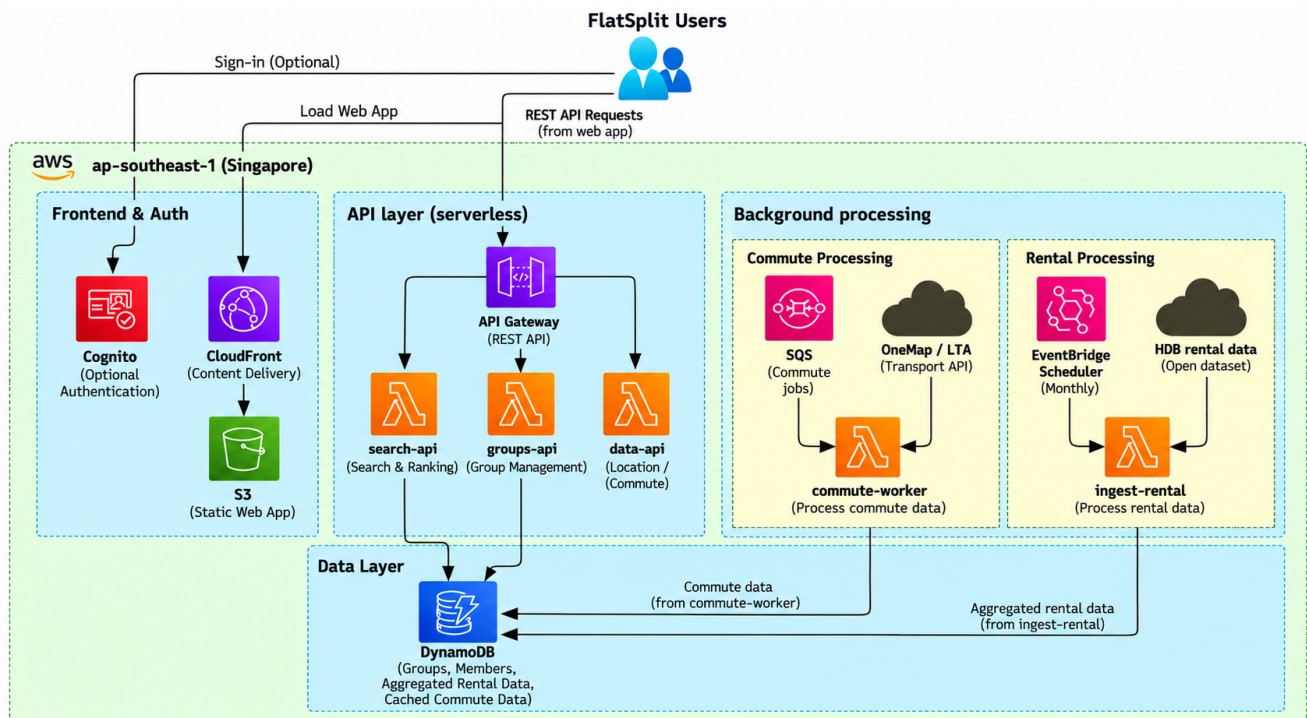
FlatSplit combines flatmates' requirements with rental and transport data to identify and fairly rank suitable rental locations.

4.1 Core Requirements

FlatSplit's prototype supports:

- **Group Search:** 2-5 flatmates submit their destination, budget and maximum commute time through a shared session.
- **Rental & Commute Data:** Historical HDB rents and public-transport commute times are processed for candidate areas.
- **Fairness-Based Ranking:** Feasible areas are ranked based on rental cost, commute time and fairness across members.
- **Recommendations:** A ranked shortlist presents suitable areas and their trade-offs.

4.2 High-Level System Design



FlatSplit uses a serverless architecture where the web application is delivered through **S3** and **CloudFront**, with optional user authentication through **Cognito**. **groups-api** manages group sessions and member requirements are stored in **DynamoDB**. **search-api** filters and ranks rental locations using cached rental and commute data. To keep searches responsive, data preparation is handled asynchronously in the background: **EventBridge** schedules periodic processing of HDB

rental data, while **SQS** queues demand-driven commute calculations for worker **Lambdas**, providing buffering and rate control when accessing external transport APIs (OneMap/LTA). The processed rental and commute data is cached in **DynamoDB** for subsequent searches, enabling low-latency retrieval, independent scaling and low idle cost.

5. Plan for Implementation

- **GitHub Tracking:** Source code and issues are maintained in the [team repository](#). The [GitHub Project](#) tracks ownership and progress, while its [timeline view](#) shows the weekly sprints and milestones.
- **Team Formation and Ownership:** The six members form three pairs covering UI and user experience, rental and commute data, and APIs and AWS. Each issue has a primary owner, reviewer, and acceptance criteria.
- **Sprint 1 (28 Sep–4 Oct):** Confirm roles, design the user journey, validate data and transit API access, agree on API contracts, and deploy the application skeleton.
- **Sprint 2 (5–11 Oct):** Build shareable groups and member input forms, implement group APIs, and publish validated rent summaries. The [shared group milestone](#) marks completion.
- **Sprint 3 (12–18 Oct):** Add destination geocoding, queued commute calculations, cached results, and progress feedback in the UI.
- **Sprint 4 (19–25 Oct):** Integrate constraint filtering and fairness-based ranking with a results map and per-member comparisons. This delivers the [usable end-to-end application](#).
- **Sprints 5–6 (26 Oct–8 Nov):** Improve usability and error handling, test complete user journeys, and measure latency, reliability, data quality, and AWS cost before the [tested release milestone](#).
- **Submission Buffer (9–13 Nov):** Freeze new features, address critical defects, review the report and slides, and rehearse the presentation. Both submissions are due **13 November 2026 at 23:59**, tracked under the [final milestone](#).

5.2 Detailed Error Recovery in Workflows

- **Transit API Failures:** Commute jobs use rate-limited retries and a dead-letter queue. Cached routes remain available; the UI marks unresolved routes as unavailable and offers a retry.
- **Rent Ingestion Failures:** Schema validation stops incompatible updates before publication. CloudWatch records the failure, while searches continue using the last validated rental snapshot.

6. Evaluation

6.1 Evaluation Dimensions

The table below summarises what we expect from the system on each dimension and the mechanism that supports it.

Dimension	Evaluation
Performance	Search latency below 500 ms. Rent and commute data are precomputed in the background, so a search only needs fast DynamoDB lookups.
Scalability	Serverless Lambda and DynamoDB On-Demand absorb peak-semester traffic spikes; SQS queues guard against external API rate limits.

Cost Efficiency	Zero idle cost. A VPC-free Lambda architecture avoids NAT Gateway fees (~US\$40/month) and uses the AWS Free Tier.
Security & Reliability	Cognito JWT authentication, least-privilege IAM roles, SQS dead-letter queues, CloudWatch logging, and DynamoDB PITR.

6.2 Deployment Risks and Mitigation

Four risks that could affect the cloud deployment, with their impact and mitigation:

Risk	Impact	Mitigation
1. External API rate limiting/downtime (OneMap / LTA DataMall)	High: commutes cannot be computed for custom destinations.	Async SQS batching at quota-matched concurrency (max 250 req/min); routes cached in DynamoDB, computed once.
2. Unexpected billing spikes	Medium: search surges could drive up Lambda/DynamoDB costs.	AWS Budgets alerts at 20/50/\$100; hard concurrency caps on Lambda; read/write limits on DynamoDB.
3. Lambda cold starts	Medium: latency could exceed the 500 ms target.	Minimal deployment packages, lean runtimes, Provisioned Concurrency during peak leasing windows.
4. Data corruption/schema issues on dataset updates	High: incorrect rent estimates shown to users.	JSON Schema validation in ETL Lambdas before any write, backed by DynamoDB PITR for restore.

6.3 User Evaluation

FlatSplit will run a usability test with 8–12 target users, including students and young working adults. Each participant completes one end-to-end flat-search task: entering group destinations, budgets, and maximum commute times, reviewing the ranked areas, and selecting a preferred location.

Metrics: task completion rate, completion time, and 5-point Likert ratings of recommendation clarity, fairness understanding, ease of use, and usefulness, plus short qualitative feedback on unclear steps and possible improvements.

Objective: determine whether users can interpret the trade-offs between rent, commute time, and fairness, and whether FlatSplit gives enough information to support a practical, balanced group rental decision.

Overall

FlatSplit is evaluated from three perspectives: system evaluation (performance, scalability, cost, security, reliability), deployment risk evaluation (API failures, cost spikes, cold starts, data errors), and user evaluation (task completion and understanding of results) — together determining whether FlatSplit is technically reliable, operationally manageable, and useful for real group flat-hunting decisions.